

UNIT 6

Working with Maven

Divya Thakur

28300

Syllabus

- Introduction to maven
- maven build lifecycle
- maven repository
- project object model
- maven dependencies
- maven plugins
- maven project structure

What is a Maven Artifact?

- In Maven, an artifact is any type of file that is used in the software development process.
- The most common of these are Java libraries, also known as 'JAR files'. Software distribution files, packages, maven project (POM) files, documentation bundles, machine learning models, and any other type of file you can think of can all be artifacts in the maven world.
- Artifacts are used in a Java program for many different purposes.
- When a JAR file artifact is used at compile time it is typically used to bring in Java library code so that code can be reused.
- Some artifacts may be packaged with the software and not used until runtime. These can include artifacts which hold data of some sort: images, machine learning models, documentation, language packs, etc.
- TLDR: Artifacts are files used by Java programs. They can also be Java programs in the case of self executing archives or other type of executable Java file.
- The most common type of artifact you'll encounter is a dependency - ie. a java library. This brings us to , Maven.

Maven Artifacts

JAR Files

Machine Learning
Models

Resource Bundles

Maven Project
Files

Images

Documents

Source and
Documentation
Archives

Videos

All Other Files

Introduction to maven

- Apache Maven is a powerful build automation tool primarily used for Java projects, but it can also be used for projects written in other programming languages.
- Maven helps manage a project's build lifecycle, dependencies, documentation, reporting, and more, providing a centralized and standardized approach to software development.
- Maven is widely used in the Java ecosystem to handle tasks like compiling code, running tests, creating JAR/WAR files, and managing dependencies.
- Developed by the Apache Software Foundation, Maven simplifies software project management by addressing the following aspects:

- Maven is a software development tool which automates dependency management by defining a project object model (POM) to abstracts the structure of a software project.
- Using POM files to represent the concept of a project, Maven provide a single model of a project that can be used by many different tools in the Java Ecosystem (IDEs, build scripts, etc).
- **Where is Maven used?**
- You can find usages of Maven in build scripts, deployment pipelines, continuous integration builds and pretty much any place where JVM based software is created and deployed.
- It's in use by teams large and small, throughout the world to compile software into JAR files, build reports and documentation, and automate many other tasks.

- Maven provides different features which make it easy to build the web-application packages we manage complex projects easily.

1. POM Files:

Project Object Model(POM) Files are XML file that contains information related to the project and configuration information such as dependencies, source directory, plugin, goals etc. used by Maven to build the project.

When you should execute a maven command you give maven a POM file to execute the commands.

Maven reads pom.xml file to accomplish its configuration and operations.

2. Dependencies and Repositories:

Dependencies are external Java libraries required for Project and repositories are directories of packaged JAR files.

The local repository is just a directory on your machine's hard drive. If the dependencies are not found in the local Maven repository, Maven downloads them from a central Maven repository and puts them in your local repository.

3. Build Life Cycles, Phases, and Goals:

A build life cycle consists of a sequence of build phases, and each build phase consists of a sequence of goals.

Maven command is the name of a build lifecycle, phase, or goal.

If a lifecycle is requested executed by giving the maven command, all build phases in that life cycle are executed also.

If a build phase is requested executed, all build phases before it in the defined sequence are executed too.

4. Build Profiles:

Build profiles a set of configuration values that allows you to build your project using different configurations. For example, you may need to build your project for your local computer, for development and test.

To enable different builds you can add different build profiles to your POM files using its profiles elements which are triggered in a variety of ways.

5. Build Plugins:

Build plugins are used to perform a specific goal. you can add a plugin to the POM file.

Maven has some standard plugins you can use, and you can also implement your own in Java.

Key Features:

- **Build Automation:** Automates the process of compiling, testing, packaging, and deploying code.
- **Dependency Management:** Automatically resolves and downloads libraries from repositories.
- **Standardized Project Structure:** Ensures consistency across projects, improving team collaboration.
- **Extensibility through Plugins:** Plugins can be added to enhance Maven's capabilities.
- **Multi-Module Projects:** Supports managing multiple interrelated projects within one build system.

Benefits of Using Maven:

- **Simplifies Project Configuration:** Uses a central file, `pom.xml`, to define all configurations.
- **Reduces Manual Effort:** Automates repetitive tasks in the build lifecycle.
- **Enhances Portability:** Ensures the project is easy to share and work on across different environments.
- **Integration with CI/CD:** Seamlessly integrates with modern DevOps tools and CI/CD pipelines.

Maven Lifecycle

- Maven is an most powerful tool to build the artifacts of the java projects by using the POM.XML file.
- A build lifecycle in Apache Maven is a prearranged set of goals and phases that are carried out in a particular order to construct and oversee a project. There are three built in build lifecycles :

1. Clean Lifecycle:

In this cycle maven will clean all the dependence which are previously stored in the local repository which have have been used for another project.

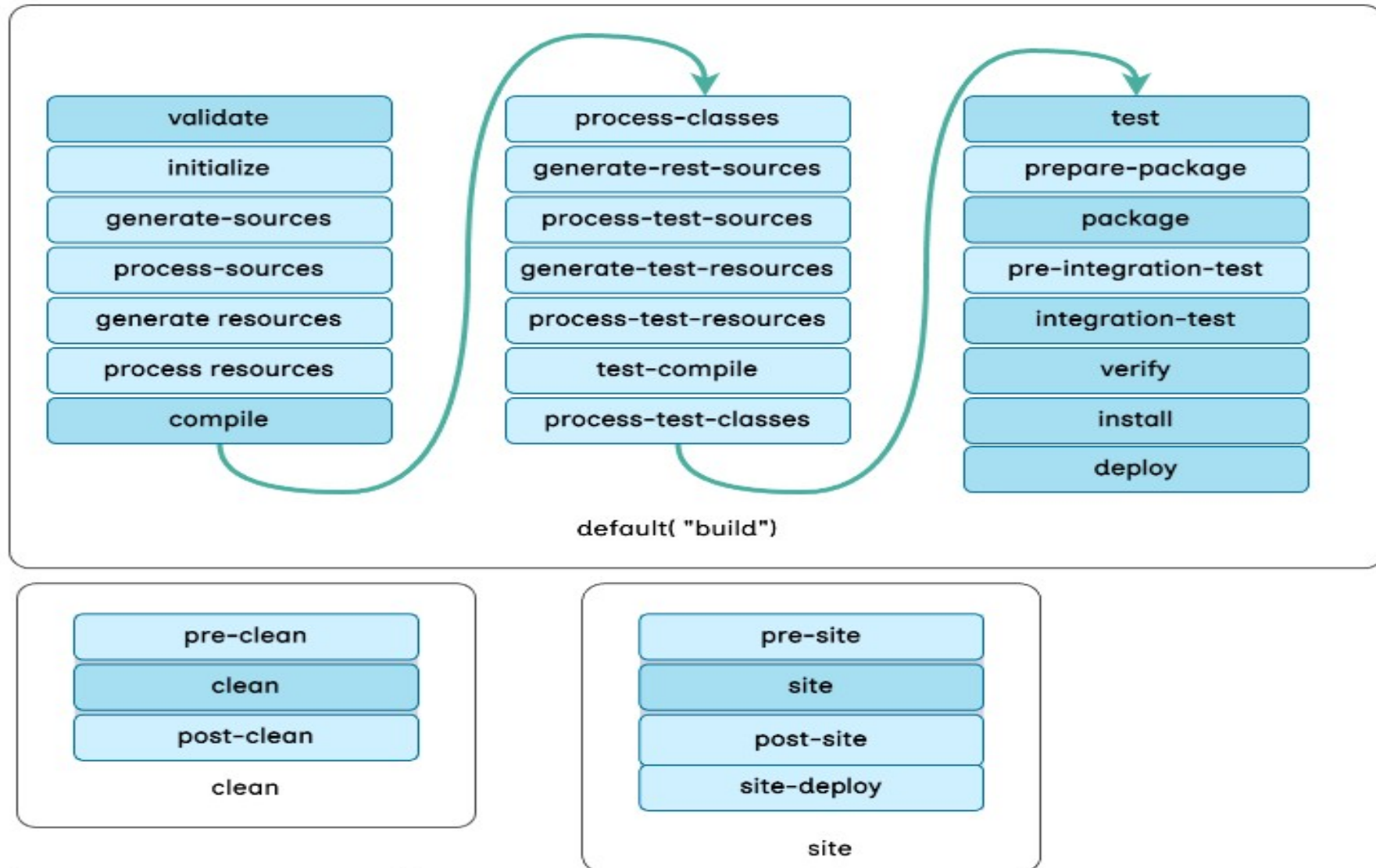
2. Default Lifecycle: Handles project deployment

This lifecycle had another sub cycles as mentioned below.

- validate: Validates the project configuration.
- compile: Compiles the source code into bytecode.
- test: Runs the tests for the project.
- package: Packages the compiled code and resources into an artifact (e.g., JAR, WAR).
- install: Installs the artifact in the local repository.
- deploy: Copies the artifact to a remote repository.

3. Site lifecycle :

Handles the creation of project's site documentation.



The clean lifecycle of Maven is a series of steps that deletes all files created by the previous build, and then completes the project's cleanup:

- Pre-clean: Completes the steps before the actual project cleaning
- Clean: Deletes all files produced by the previous build, including compiled files from the output directory
- Post-clean: Completes the steps required to finish the project's cleanup
- **The mvn clean command deletes the target directory to clean the Maven project.**

Here's a brief description of each phase in the default lifecycle:

- **validate:** It validates the project is correct and all necessary information is available.
- **compile:** It compiles the source code of the project.
- **test:** It tests the compiled source code using a suitable unit testing framework.
- **package:** It takes the compiled code and packages it in its distributable format, such as a JAR file.
- **verify:** To run any checks to verify the package is valid and meets quality criteria.
- **install:** To install the package into the local repository, for use as a dependency in other projects.
- **deploy:** To copy the final package to the remote repository for sharing with other developers and projects.

The Maven site lifecycle is a set of phases that are executed to create a project's site documentation:

- Pre-site: A phase that is executed before the site phase
- Site: A phase that creates the project's site documentation
- Post-site: A phase that is executed after the site phase
- Site-deploy: A phase that executes all of the other phases in the site lifecycle

JAR Files

- JAR stands for Java ARchive.
- It's a file format that's used to bundle many files into one, and is based on the ZIP file format. JAR files are typically used to aggregate Java class files, metadata, and resources into one file for distribution.
- In Maven, you can use the Apache Maven JAR Plugin to build JAR files:
- Create a pom.xml file
- Locate the Java source files in src/main/java
- Locate extra resources in src/main/resources
- Use the command mvn package to create the JAR file
- The JAR file will contain the compiled Java class files and the files from src/main/resources.
- You can also use other Maven plugins to generate JAR files with dependencies, which are called Fat Jars. Fat Jars are useful for building executable JAR files. For example, you can use the Maven Assembly plugin to combine project output into a single distributable archive that contains dependencies, modules, and other files.

WAR Files

- A WAR file, or Web Archive file, is a packaged web application that contains resources used to build a web application:
- **Java classes**
- **JavaServer Pages (JSP)**
- **Java servlets**
- **XML files**
- **Tag libraries**
- **Static web pages**
- **Static images**
- WAR files are similar to JAR files, but they also include a web deployment descriptor file, which is an XML file that contains: Deployment information, MIME types, Session configuration details, and Other settings for the web application.
- WAR files can be used to import a web application into a web server.
- The web deployment descriptor file is shared with developers, assemblers, and deployers in a Java EE environment.
- You can install Apache Tomcat from a WAR file.
- You can import WAR files.

- A WAR file is used to deploy a Java EE web application in an application server. Inside a WAR file, all the web components are packed into one single unit. These include JAR files, JavaServer Pages, Java servlets, Java class files, XML files, HTML files, and other resource files that we need for web applications.

What Is Maven Repository?

- Maven repositories are directories of packaged JAR files with some metadata.
- The metadata are POM files related to the projects each packaged JAR file belongs to, including what external dependencies each packaged JAR has.
- This metadata enables Maven to download dependencies of your dependencies recursively until all dependencies are download and put into your local machine.
- Maven has three types of repository :Maven searches for dependencies in this repositories. First maven searches in Local repository then Central repository then Remote repository if Remote repository specified in the POM.

Local Repository



Your Pc

Remote
Repository



Some Server

Central Repository



Internet

- **Local repository-**

- A local repository is a directory on the machine of developer.
- This repository contains all the dependencies Maven downloads.
- Maven only needs to download the dependencies once, even if multiple projects depends on them (e.g. ODBC).
- By default, maven local repository is user_home/m2 directory.

- **Central repository-**

- The central Maven repository is created Maven community.
- Maven looks in this central repository for any dependencies needed but not found in your local repository.
- Maven then downloads these dependencies into your local repository.
- You can view central repository by this link.

- **Remote repository-**

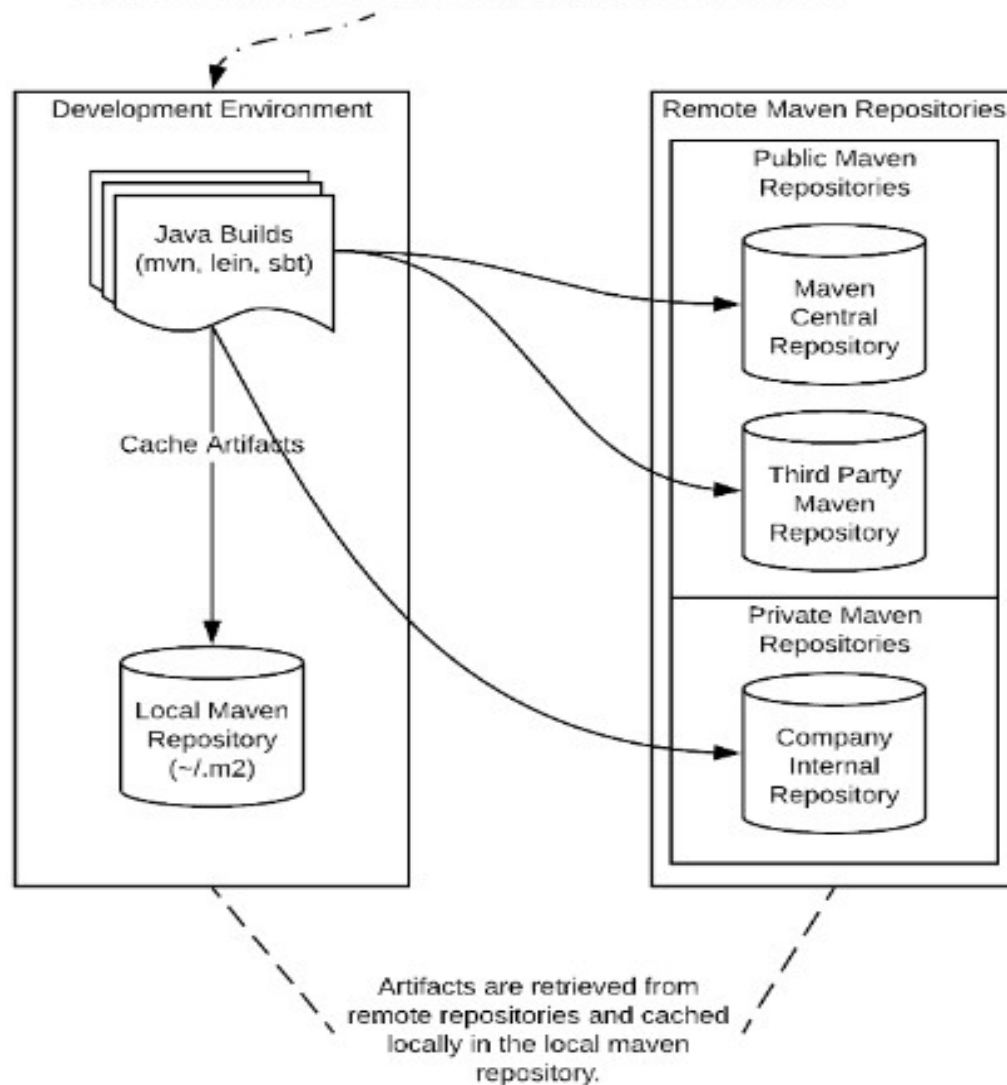
- Remote repository is a repository on a web server from which Maven can download dependencies. It is often used for hosting projects internal to the organization.
- Maven then downloads these dependencies into your local repository.

- Maven can add all the dependencies required for the project automatically by reading pom file.
- One can easily build their project to jar, war etc. as per their requirements using Maven.
- Maven makes easy to start project in different environments and one doesn't need to handle the dependencies injection, builds, processing, etc.
- Adding a new dependency is very easy. One has to just write the dependency code in pom file.
- Maven needs the maven installation in the system for working and maven plugin for the ide.
- If the maven code for an existing dependency is not available, then one cannot add that dependency using maven.
- When there are a lot of dependencies for the project. Then it is easy to handle those dependencies using maven.
- When dependency version update frequently. Then one has to only update version ID in pom file to update dependencies.
- Continuous builds, integration, and testing can be easily handled by using maven.
- When one needs an easy way to Generating documentation from the source code, Compiling source code, Packaging compiled code into JAR files or ZIP files.

Maven Repositories

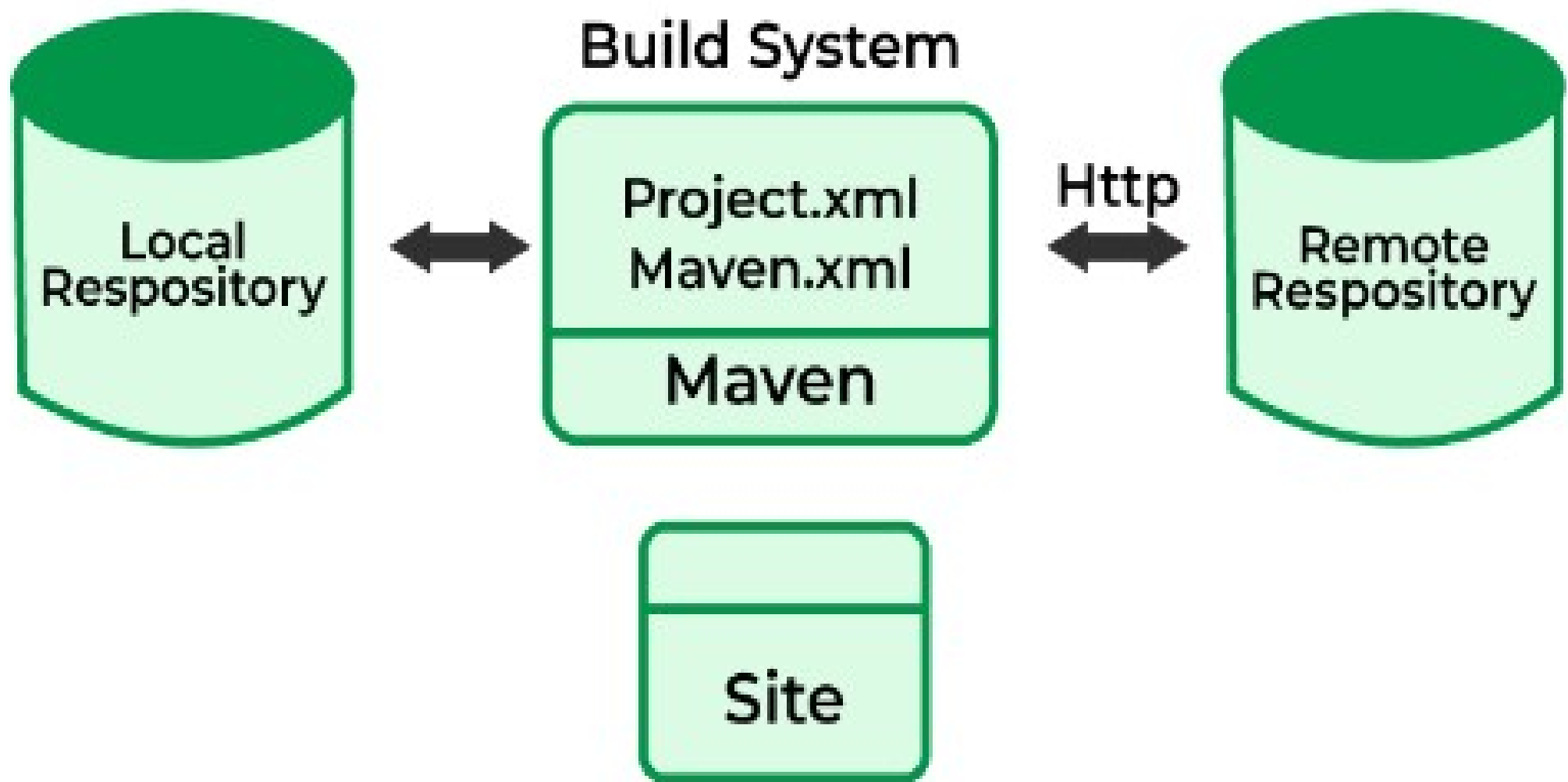
This diagram illustrates how maven repositories fit within the software development lifecycle.

A Maven Development Environment can be on a local developer machine, or part of a continuous integration and deployment environment



What Is Maven Architecture?

- Maven repository is a place where the maven artefacts or dependencies of the JAR file is going to store which are written in the file called the POM.XML.
- POM.XML contains the Java classes, resources, and other dependencies.
- Maven reads the pom.xml file.
- Maven downloads the dependencies defined in the pom.xml file into the local repository from the central or remote repository.
- Maven executes the life cycles, phases, goals, and plugins defined in the pom.xml file.



- Maven is a build tool that automates the process of developing and building applications.
- It's written in Java and uses the Project Object Model (POM) to:
 - Create projects from templates
 - Manage project details, such as change logs, dependencies, and unit test reports
 - Preserve project metadata in a central repository
 - Simplify and standardize the building process

Uses of Maven

- Create projects: Maven creates Java projects from templates and a project directory.
- Manage project details: Maven manages project details in the form of project reports and project information.
- Use plugins: Maven uses plugins to build projects.
- Help with project workflow: Maven helps with project workflow such as release, issue management, specification, execution, and reporting of unit tests.
- Use convention over configuration: Maven uses convention over configuration, which means that developers only need to create a Maven project.
- Handle multi-module projects: Maven uses the reactor to handle multi-module projects. The reactor collects all the available modules to build, sorts them into the correct build order, and builds them in order.
- Use the Maven build lifecycle: Maven has three built-in lifecycles: default, clean, and site. The default lifecycle handles the regular build process, the clean life cycle removes generated artifacts, and the site life cycle generates project documentation and reports.

Importance and uses of Maven

- We can easily build a project using maven.
- We can add jars and other dependencies of the project easily using the help of maven.
- Maven provides project information (log document, dependency list, unit test reports, etc.)
- Maven is very helpful for a project while updating the central repository of JARs and other dependencies.
- With the help of Maven, we can build any number of projects into output types like the JAR, WAR, etc without doing any scripting.
- Using maven we can easily integrate our project with a source control systems (such as Subversion or Git).
- Maven also helps in managing the project's build lifecycle, including tasks like compiling, testing, packaging, and deploying the code.
- Maven provides a standard project structure, making it easy for developers to understand the layout of the project and locate specific files.
- Maven supports multi-module projects, allowing developers to work on multiple related projects simultaneously and manage their dependencies efficiently.
- Maven plugins can be used to add additional functionality to the build process, such as code coverage analysis, static code analysis, and more.
- Maven is highly customizable, allowing developers to configure the build process to meet their specific needs and requirements.
- Maven simplifies the process of managing project dependencies, ensuring that the correct versions of libraries and frameworks are used throughout the project.

Maven's architecture works by:

- **Configuring Maven**

The first step is to configure Maven in a pom.xml file. This file includes all the configurations that Maven needs.

- **Downloading dependencies**

Maven downloads dependencies from the central repository into the local repository.

- **Using default settings**

Maven provides default settings so that users don't need to add every configuration in the pom.xml.

- **Using conventions**

Maven uses conventions for the build procedure, so only exceptions need to be written down.

- **Using a local cache**

Maven dynamically downloads Java libraries and plugins from repositories and stores them in a local cache.

- **Using a build lifecycle**

Maven follows a build lifecycle to deploy and distribute the target project.

- **Using dependency management**

Maven's dependency management examines pom files for plugins and dependencies, and then accesses specific repositories.

- **Using packaging**

Users can set the packaging for their project via the <packaging> element in the pom.xml file.

- **Using the Super POM**

The Super POM is Maven's default POM, and all POMs extend it unless explicitly set.

- **Maven is a tool that helps users develop projects by:**

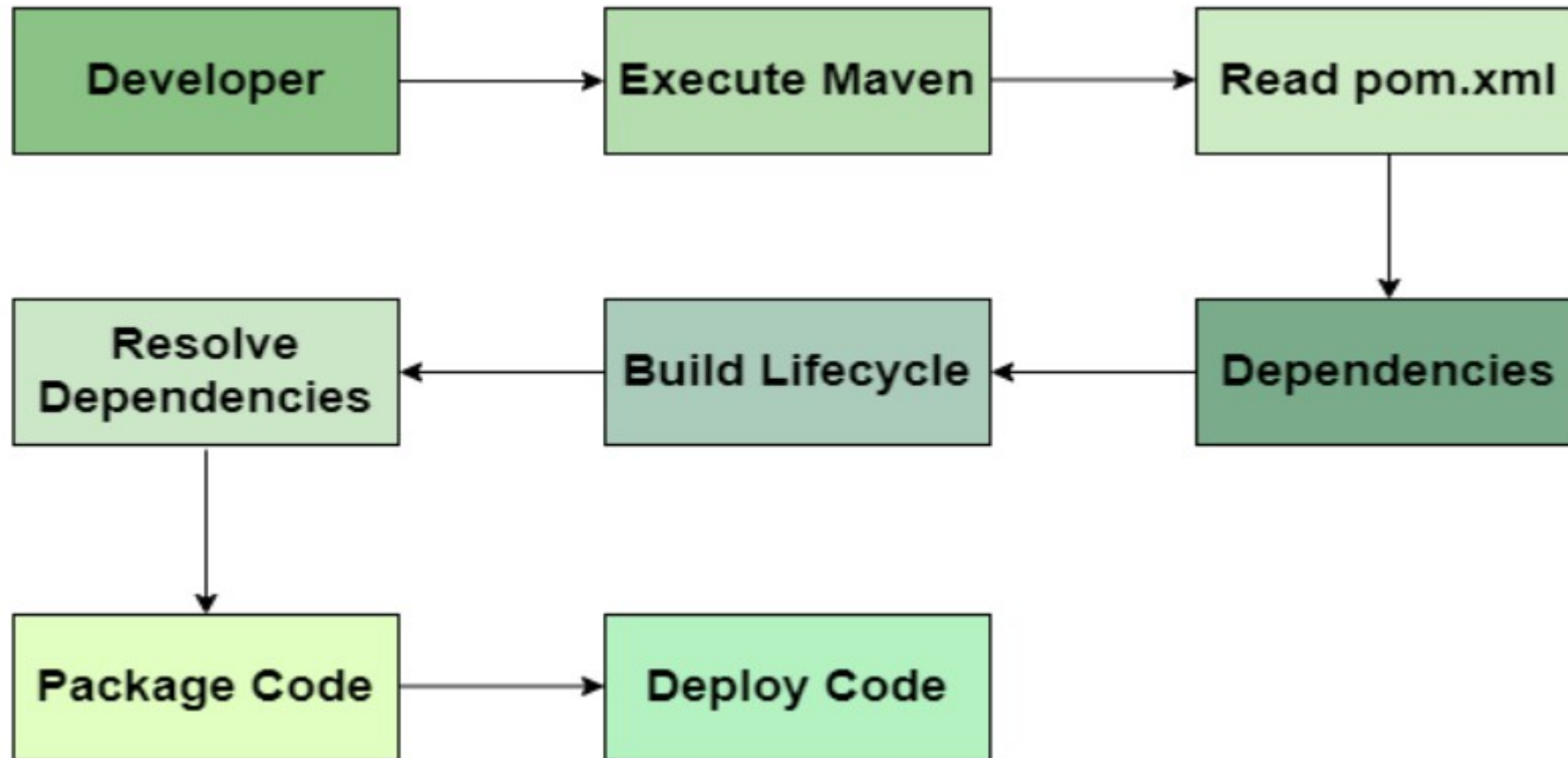
1. Describing how software is built and dependencies
2. Providing project information, such as log documents, dependency lists, and unit test reports
3. Facilitating the acquisition of plugins and dependencies
4. Encouraging better development practices

Project Object Model (POM)

- POM stands for Project Object Model, and it's a fundamental component of Maven, a build automation tool for Java projects.
- A POM is an XML file that contains information about a Maven project, including its configuration and dependencies.
- Maven uses the POM to build a project by reading it for configuration information and then executing the desired goal or task.
- A POM can include information such as the project's version, description, developers, dependencies, build profiles, and more.
- The POM file is located in the root directory of the project it belongs to.
- In larger projects, a project can import managed dependencies from other projects by declaring a POM artifact as a dependency with a scope of "import".

- POM means Project Object Model is key to operate Maven.
- Maven reads pom.xml file to accomplish its configuration and operations.
- It is an XML file that contains information related to the project and configuration information such as dependencies, source directory, plugin, goals etc. used by Maven to build the project.

Maven Workflow



Workflow of POM

- Initialization: The Maven reads the pom.xml file and initializes the build process.
- Dependency Resolution: Downloads the Specified dependencies from remote repositories.
- Build Life cycle Execution: Executes the build life cycle phases like compile, test, package, verify and other phases.
- Plugin Execution: Runs the configured plugins for various tasks like code analysis and other tasks.
- Packaging: Packages the compiled code into specified format like JAR, WAR and other formats.
- Deployment: Deploys the packaged code to a remote repository or server.

The sample of pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId> com.project.loggerapi </groupId>
  <artifactId>LoggerApi</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <!-- Add typical dependencies for a web application -->
  <dependencies>
    <dependency>
      <groupId>org.apache.logging.log4j</groupId>
      <artifactId>log4j-api</artifactId>
      <version>2.11.0</version>
    </dependency>
  </dependencies>

</project>
```

Key Components of a POM File:

- **Project Coordinates:**
 - <groupId>: Defines the group or organization to which the project belongs.
 - <artifactId>: The unique name of the project.
 - <version>: The specific version of the project.
- **Build Configuration:**
 - <build>: Contains the build configuration for the project, such as the source directory, test source directory, and output directory.
- **Dependencies:**
 - <dependencies>: Contains the list of dependencies required by the project. Each dependency is defined by its groupId, artifactId, version, and scope.
- **Plugins:**
 - <plugins>: Defines the list of plugins that are used in the build process.
- **Repositories:**
 - <repositories>: Defines the list of remote repositories from which dependencies and plugins can be downloaded.
- **Profiles:**
 - <profiles>: Allows the definition of different configurations for different environments, such as development, testing, and production.

uses of pom

- **Dependency Management:** It can declares and manages project dependencies.
- **Build Configuration:** It can specifies plugins and their configuration for various build tasks.
- **Project Information:** It contains metadata like project version, description, developers and other things.
- **Reporting:** Defines reporting plugins for generating Project reports like Java Docs.
- **Build Profiles:** It configures Different build profiles for various environments.

- The Maven Tools provides lot of advantages listed below :
- Standardization: Provides a standard way to manage project builds.
- Dependency Management: Automatically handles dependency resolution and version conflicts.
- Reproducibility: It Builds are reproducible by using versioned dependencies.
- Integration with CI/CD: Easily integrates with continuous integration and delivery pipelines.
- Extensibility: Supports a wide range of plugins to extend build functionality.

Maven Dependencies

- A dependency is an external object or library on which a Maven project depends for compilation, build, and execution.
- In Java, a dependency is usually a Java Archive (JAR) file that contains compiled code or other resources that are required for the project.
- Maven defines dependencies in the POM (Project Object Model) file.
- The POM defines the external artifacts and the versions of those artifacts.
- When you build your Maven project, your Maven build system will automatically download and include these dependencies.
- This ensures that your project will have access to the classes and resources you need during build and execution.
- Importance of Dependencies : Dependencies help a project be modular, maintainable, and reusable by allowing developers to seamlessly integrate external functionality into their projects.

- In Maven, a dependency is an archive file like JAR, ZIP, etc., that your project needs to compile, build, test, and run.
- These project dependencies are defined in the pom.xml file.
- When executing a maven build or goal, these project dependencies are resolved and then fetched from the local repository.
- You can also install the dependencies manually.
- There are two types of Maven dependencies:
- **Direct dependency:** It allows you to explicitly include dependencies in your project using <dependency> tags.
- **Transitive dependency:** These dependencies are required by direct dependencies Maven automatically includes in your project.
- **To list all your project's and transitive dependencies, use the `mvn dependency:tree` command.**

Maven plugins

- Maven plugins are essential components in the Apache Maven build system designed to extend its functionality.
- They perform tasks such as compilation, testing, packaging, deployment, and others.
- In Maven, the plugins are divided into two types.
 - Build Plugins
 - Reporting Plugins

Maven Build Plugins

- These plugins are executed during the build process and these are defined in the <build> section in the pom.xml file.
- For example, Compiler Plugin, Surefire Plugin, and Assembly Plugin.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.8.1</version>
      <configuration>
        <source>1.8</source>
        <target>1.8</target>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>2.22.2</version>
    </plugin>
  </plugins>
</build>
```

Maven Reporting Plugins

- These plugins are used for generating reports about the project.
- These are defined in the `<reporting>` section in the pom.xml file.
- Examples include the Surefire Report Plugin for generating test reports and Javadoc Plugin for generating API documentation.

```
<reporting>
  <plugins>
    <!-- Surefire Report Plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-report-plugin</artifactId>
      <version>2.22.2</version>
    </plugin>

    <!-- Javadoc Plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-javadoc-plugin</artifactId>
      <version>3.3.1</version>
      <configuration>
        <source>1.8</source>
      </configuration>
    </plugin>

    <!-- Checkstyle Plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-checkstyle-plugin</artifactId>
      <version>3.1.2</version>
      <configuration>
        <configLocation>google_checks.xml</configLocation>
      </configuration>
    </plugin>
  </plugins>
</reporting>
```

Different types of Maven Plugins

- Compiler Plugin: It is used for Compiles the project source code.
- Surefire Plugin: It is used for Run the Project unit Tests.
- Failsafe Plugin: It is used for Runs the project integration tests.
- JAR Plugin: It is used for Packages the compiled code into a JAR file.
- War Plugin: It is used for Packages the compiled code into a WAR file.
- Assembly Plugin: It is used for Creates distributions from the project.
- Javadoc Plugin: It is used for Generates Javadoc documentation for the project.
- Checkstyle Plugin: It is used for Checks the code against coding standards.

Working Process of Plugins in a Maven Project

- Define the project configuration and project structure and build plugins configurations in the POM file
- Now configure the required plugins in the maven project.
- After that maven build process is started by help of different phases in the maven build like validate, compile, test, package, verify, install, and deploy.

Javadocs Plugin

- Javadoc is a documentation generator created by Sun Microsystems for the Java language for generating API documentation in HTML format from Java source code.
- The Javadoc Plugin uses the Javadoc tool to generate javadocs for the specified project.
- The Javadoc Plugin generates javadocs using the Javadoc tool. The following examples describe the basic usage of the Plugin.
- The Javadoc generation can be skipped by setting the property `maven.javadoc.skip` to `true`.

Generate Javadocs As Part Of Project Reports

- To generate javadocs as part of the site generation, you should add the Javadoc Plugin in the `<reporting>` section of your pom.
- When you execute `mvn site`, the javadocs will be generated and included in the generated site. A link to the javadocs will be added in the Project Reports menu.

Generate Standalone Javadocs

- To generate standalone javadocs for the project, you could add the Javadoc Plugin in the <build> section of your pom (if no configuration defined, the plugin uses default values).
- And execute command:

```
mvn javadoc:javadoc
```

Maven Project Structure

- When you create a Maven project, the directory structure that is created includes the project object model (POM) file.
- The Maven project wizard creates src and target folders under the main project folder.
- As the name suggests, all source files go under src. However, Java package structure starts under the main folder.
- By convention, Maven expects Java source files under the java folder.
- Therefore, create a java folder under src/main.
- The Java package structure starts from the java folder, that is, src/main/java/<java-packages>.
- Web content such as HTML, JS, CSS, and JSP goes in the webapp folder under src/main.
- Compiled classes and other output files generated by the Maven build process are stored in the target folder.

Maven Project Structure

```
project-root/  
├── src/  
│   ├── main/  
│   │   ├── java/  
│   │   ├── resources/  
│   │   └── webapp/ (optional)  
│   └── test/  
│       ├── java/  
│       └── resources/  
├── target/  
└── pom.xml
```

Key Directories in Maven Structure

a. **src/**

- This is the main source directory where all your code and resources reside.
 - **src/main/:**
 - Contains production code (the code used in the actual application).
 - **src/main/java/:**
 - The directory for Java source files.
 - Example: com/example/MyApp.java
 - **src/main/resources/:**
 - Contains resources required for the application (e.g., configuration files like application.properties, XMLs, or files loaded at runtime).
 - These files are included in the compiled JAR/WAR.
 - Example: src/main/resources/log4j.properties.

- **src/main/webapp/ (optional):**
 - Used for web applications (WAR packaging).
 - Contains web resources like HTML, JSP files, CSS, JS, and WEB-INF configurations.
 - Example: src/main/webapp/WEB-INF/web.xml.
- **src/test/:**
 - Contains test code and test-related resources.
- **src/test/java/:**
 - Test Java source files (e.g., JUnit or TestNG test cases).
 - Example: com/example/MyAppTest.java.
- **src/test/resources/:**
 - Test-specific resources like test configuration files or mock data.
 - Example: test-database.properties.

b. target/

- The target/ directory is where Maven places compiled files, packaged artifacts (JAR/WAR files), and other build outputs.
- Examples of contents:
 - classes/: Compiled .class files from src/main/java.
 - test-classes/: Compiled test .class files from src/test/java.
 - app.jar or app.war: The packaged application.

c. pom.xml

- The Project Object Model (POM) file is the heart of a Maven project.
- It defines:
 - Project Information:
 - Name, version, description, and more.
 - Dependencies:
 - Libraries and frameworks your project uses.
 - Example: Spring Boot, Hibernate, etc.
 - Build Plugins:
 - Tools like compilers, package managers, etc.
 - Example: Maven Compiler Plugin for setting Java versions.
 - Project Structure and Packaging:
 - Packaging type (jar, war, etc.).

Benefits of Maven's Standard Project Structure

- Consistency:
 - Teams follow the same layout, making projects easier to understand and maintain.
- Tool Support:
 - Many IDEs and tools (e.g., IntelliJ IDEA, Eclipse, Jenkins) recognize and work seamlessly with Maven projects.
- Convention Over Configuration:
 - By following conventions, you reduce the need for extensive configuration.
- Simplified Builds:
 - Maven plugins assume this directory structure, minimizing manual setup.

Installation

Detailed installation of Maven for Windows

Ensure Java is installed: Maven requires Java to run. Verify your Java installation with:

java -version

If Java is not installed, download and install the Java JDK from Oracle or OpenJDK.

Step 1: Download Maven

- Go to the Apache Maven Download Page.
- Under the Files section, click the link to download the latest Binary zip archive.
- File : apache-maven-3.9.9-bin.zip
- Link : <https://maven.apache.org/download.cgi>

Step 2 : Extract the Archive

- Locate the downloaded zip file (**apache-maven-3.9.9-bin.zip**) in your Downloads folder.
- Right-click the file and choose Extract All or use a tool like WinRAR to extract it.
- Extract the contents to a directory, for example:
 - **C:\Program Files\Apache\Maven**
- After extraction, the folder structure will look something like:
 - **C:\Program Files\Apache\Maven\apache-maven-3.9.9**

Step 3: **Set Environment Variables**

- Open the System Properties:
 - Press Windows + R, type sysdm.cpl, and press Enter.
- In the System Properties window:
 - Go to the Advanced tab.
 - Click on Environment Variables.
- Add MAVEN_HOME:
 - In the Environment Variables window, under System Variables, click New.
 - Enter the following:
 - Variable Name: MAVEN_HOME
 - Variable Value: The directory where Maven is extracted (e.g., C:\Program Files\Apache\Maven\apache-maven-x.x.x).

- Add Maven to PATH:
 - Under System Variables, find and select the Path variable, then click Edit.
 - In the Edit Environment Variable window:
 - Click New and add : : %MAVEN_HOME%\bin
- Save and Close:
 - Click OK to close all windows and save your changes.

Step 4: Verify the Installation

- Open a Command Prompt:
 - Press Windows + R, type cmd, and press Enter.
- Run the following command to verify Maven is installed:
`mvn -version`
- Output :
 - Apache Maven 3.9.9
 - Maven home: C:\Program Files\Apache\Maven\apache-maven-3.9.9
 - Java version: 1.x.x
 - Java home: C:\Program Files\Java\jdk-x.x.x
 - Default locale: en_US, platform encoding: UTF-8

Step 5: Troubleshooting

- If mvn is not recognized:
 - Double-check the MAVEN_HOME variable and Path entries for typos.
 - Ensure the %MAVEN_HOME%\bin path is correctly appended to the Path variable.
 - Restart the Command Prompt or your computer to apply the changes.